Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ” Python Web”

A efectuat : st. gr.SI – 264, Tîltu Daniel

A verificat : asistent universitar, Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Mersul Lucrarii

Capitolul I: Exercitiile 1-8

```
4   import time
5   import requests
6   import urllib.request
```

FIG. 1.1 — Importul bibliotecilor

time(biblioteca standard pentru a masura sau pentru a face pauze in cod), requests(biblioteca externa pentru a trimite cereri catre site-ri sau API-uri), urllib.request(ca si requests, doar ca e standard si ma greoaie)

```
 8   BASE_URL = "https://cybercor.org"
 9   ECHO_URL = "https://httpbin.org"
10   TIMEOUT = 10
```

FIG. 1.2 — Declararea

2 stringuri cu doua adrese si un int egal cu 10

```
 9   print ("Ex 1. Versiune requests: ", requests.__version__)

Problems  Output  Debug Console  Terminal  Ports                Code

Ex 1. Versiune requests:  2.34.2
```

FIG. 1.3 — Version

Afisare versiunii bibliotecii requests

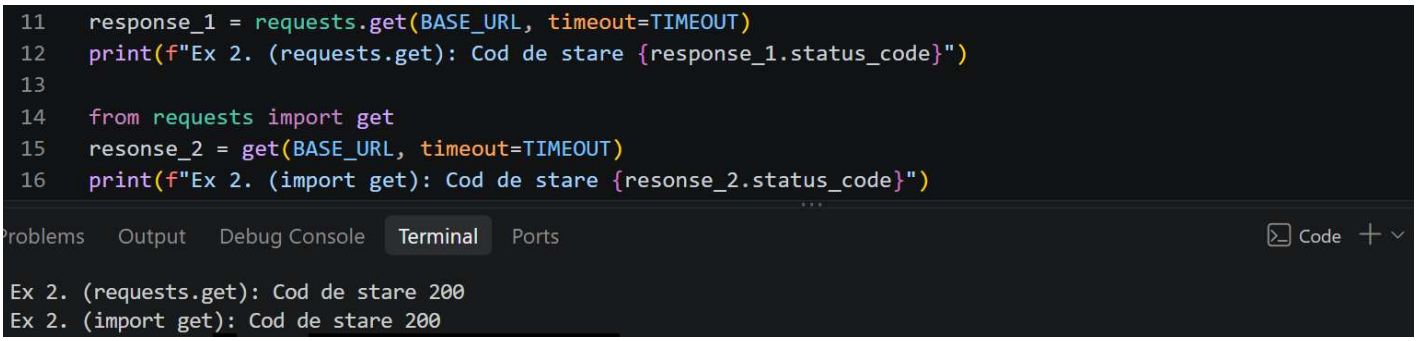


FIG. 1.4 — GET

Face 2 cereri care sunt salvate in response 1 si 2, apoi afisate

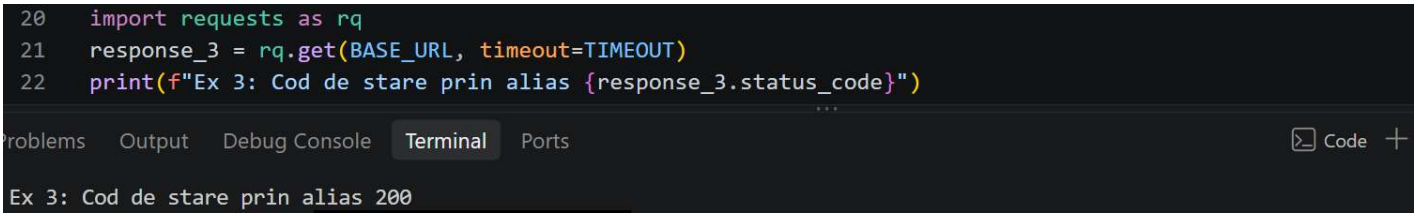


FIG. 1.5 — Redenumire

Face aceasi cerere doar ca, din cauza la import asta i-a dat putin alt nume

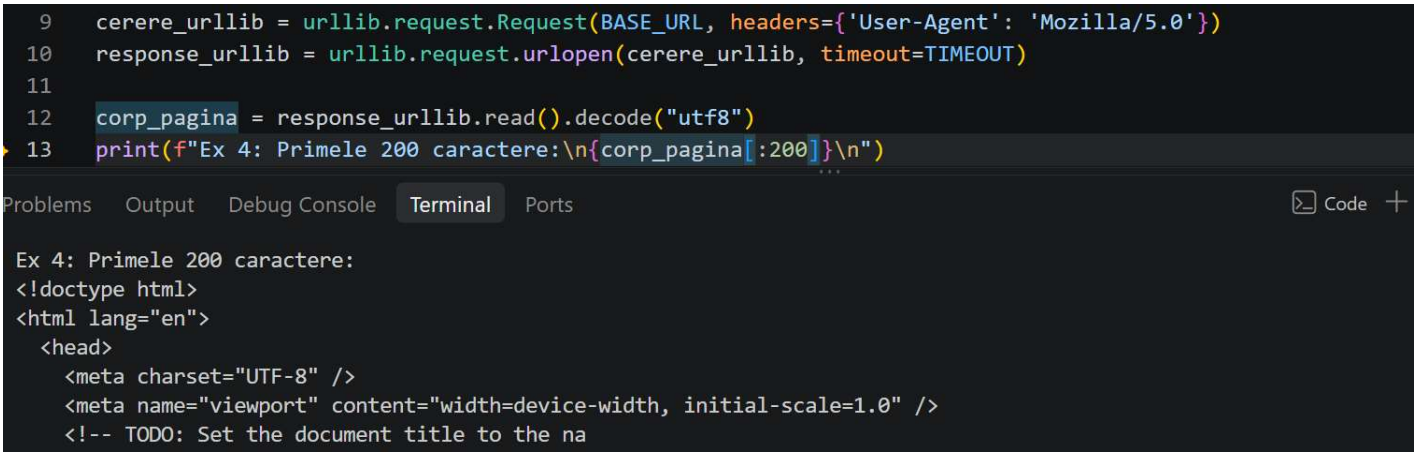


FIG. 1.6 — Read

Aceeasi cerere, daor cu mici modificari in user-agent, decode si afisare nu la status code ci la ceva codul HTML

```
9   print(dir(requests))
```

Problems Output Debug Console Terminal Ports Code

```
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedR
equest', 'ReadTimeout', 'Request', 'RequestException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects
', 'URLRequired', '__all__', '__author__', '__author_email__', '__build__', '__builtins__', '__cake__', '__copyright__', '__description
__', '__doc__', '__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__', '__url__', '__
version__', '_check_cryptography', '_internal_utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', '
charset_normalizer_version', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logg
ing', 'models', 'options', 'packages', 'patch', 'post', 'put', 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', '
urllib3', 'utils', 'warnings']
```

FIG. 1.7 — Intim

Cu ajutorul la dir am afisat tot ce este in requests

```
9   help(requests.get)
```

Problems Output Debug Console Terminal Ports Code

```
Help on function get in module requests.api:

get(
    url: _t.UriType,
    params: _t.ParamsType = None,
    **kwargs: Unpack[_t.GetKwargs]
) -> Response
    Sends a GET request.
```

FIG. 1.8 — help

Cu help poti vedea ce si pentru ce e o anumita functie

```
9   start_time = time.perf_counter()
10  response_chrono = requests.get(BASE_URL, timeout=TIMEOUT)
11  end_time = time.perf_counter()
12
13  timp_masurat = end_time - start_time
14  timp_raportat = response_chrono.elapsed.total_seconds()
15
16  print(f"Ex. 7: Timp masurat manual: {timp_masurat:.4f} secunde")
17  print(f"Ex. 7: Timp raportat de requests: {timp_raportat:.4f} secuntde")
```

Problems Output Debug Console Terminal Ports Code

```
Ex. 7: Timp masurat manual: 0.1049 secunde
Ex. 7: Timp raportat de requests: 0.1015 secuntde
```

FIG. 1.9 — Timp de raspuns

Am masurat timpul de raspuns manual si cu ajutorul la ceva functie. Diferenta a fost minima

```
5   try:
6       import bs4
7       print("Ex 8: bs4 este instalat!")
8   except:
9       print("Ex 8: Instalați modulul cu: pip install beautifulsoup4")
```

Problems Output Debug Console Terminal Ports Code

```
Ex 8: bs4 este instalat!
```

FIG. 2.0 — try & except

Verifica daca am instalat bs4, daca de eruare merge la blocul cu except

Capitolul II: Exercitiile 9-20

```
17  print("--- Exercițiul 9 ---")
18  print("Status code:", response.status_code) # atribut
19  print("Este OK?:", response.ok)             # atribut
20  print("URL:", response.url)                 # atribut
21  print("Encoding:", response.encoding)       # atribut
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 9 ---
Status code: 200
Este OK?: True
URL: https://cybercor.org/
Encoding: utf-8
```

FIG. 2.0 — Atributele

```
26  print("\n--- Exercițiul 10 ---")
27  response.raise_for_status()
28
29  try:
30      bad_response = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
31      bad_response.raise_for_status()
32  except requests.HTTPError:
33      print("Pagina nu a fost găsită (Eroare 404).")
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 10 ---
```

FIG. 2.1 — 404

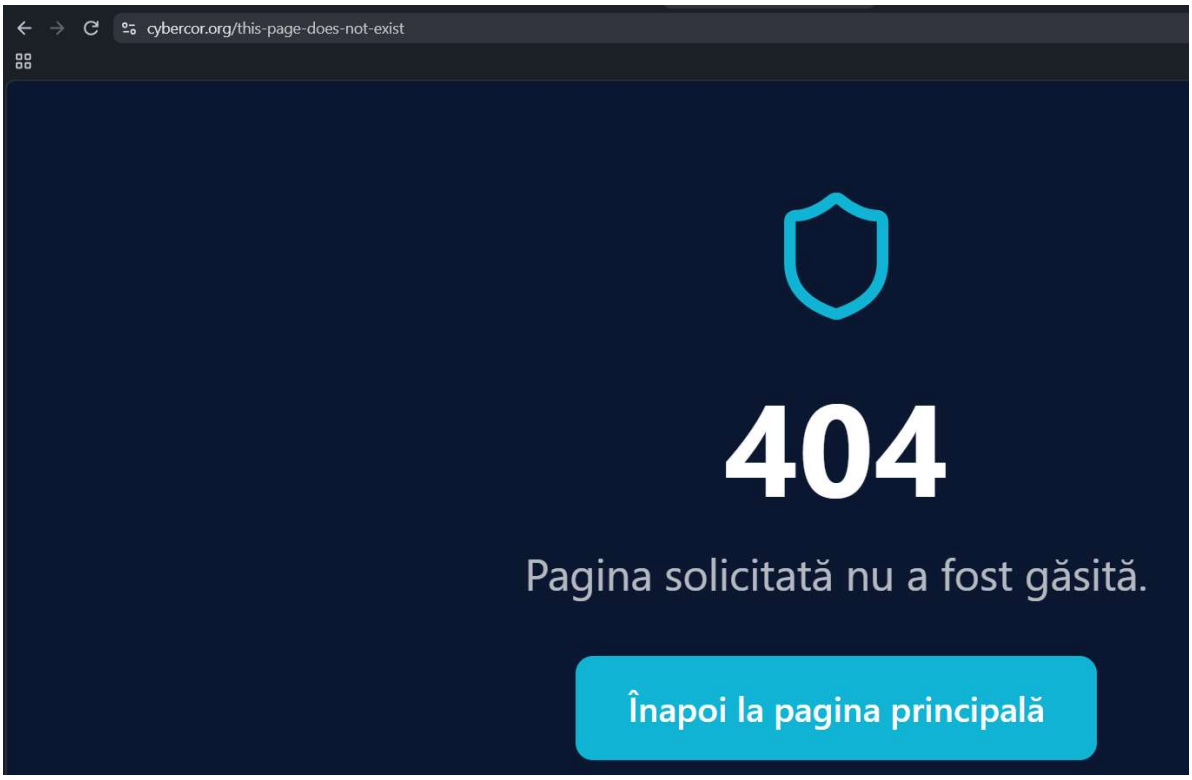


FIG. 2.2 — 404

```
40  print("\n--- Exercițiul 11 ---")
41  for nume, valoare in response.headers.items():
42      print(f"{nume}: {valoare}")
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 11 ---
Date: Mon, 19 Oct 2026 15:27:55 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cache-Control: no-cache
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=0seAR0n
FkzDduAfUSDUCxujxzb46NrUPCQ58X1%2FQuUAbyVrJcguN%2FSTHsnB%2BRXezrBzldVffZTF%2F7EZ8aa2eXODO2QMFwiMFmjVDcfUCDgg6YLkGIA
w%2F16dfBG3v8WI%3D"}]}
Last-Modified: Wed, 14 Oct 2026 19:03:10 GMT
X-Powered-By: Express
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
cf-cache-status: DYNAMIC
Content-Encoding: zstd
Server: cloudflare
CF-RAY: a4d0cf410eaf5e2e-KIV
alt-svc: h3=":443"; ma=86400
```

FIG. 2.3 — Haders

Afiseaza antetii

```
48  print("\n--- Exercițiul 12 ---")
49  print("Server:", response.headers.get("Server", "lipsește"))
50  print("Content-Type:", response.headers.get("Content-Type", "lipsește"))
51  print("content-type (litere mici):", response.headers.get("content-type", "lipsește"))
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 12 ---
Server: cloudflare
Content-Type: text/html; charset=utf-8
content-type (litere mici): text/html; charset=utf-8
```

FIG. 2.4 — Antetele specificate

Extragerea unor antete HTTP specifice folosind metoda `.get()`

```
58  print("\n--- Exercițiul 13 ---")
59  aparitii_cyber = response.text.lower().count("cyber")
60  print(f"Cuvântul 'cyber' apare de: {aparitii_cyber} ori")
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 13 ---
Cuvântul 'cyber' apare de: 9 ori
```

FIG. 2.5 — Counter

Numara de cate ori a aparut “cyber” pe pagina

```
67  print("\n--- Exercițiul 14 ---")
68  html = response.text
69  start = html.find("<title>") + len("<title>")
70  end = html.find("</title>")
71  titlu = html[start:end].strip()
72  print("Titlul paginii este:", titlu)
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 14 ---
Titlul paginii este: CYBERCOR
```

FIG. 2.6 — <title>

Gaseste inceputul titlului si sfarsitul, dupa il afiseaza

```
77  print("\n--- Exercițiul 15 ---")
78  linii = html.splitlines()
79  print("Numărul total de linii HTML:", len(linii))
80  cea_mai_lunga = max(linii, key=len)
81  print("Lungimea celei mai lungi linii:", len(cea_mai_lunga))
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 15 ---
Numărul total de linii HTML: 32
Lungimea celei mai lungi linii: 218
```

FIG. 2.7 — Counter

Afiseaza numarul de linii cod HTML, si ce mai lunga linie

```
86  print("\n--- Exercițiul 16 ---")
87  if response.url.startswith("https://"):
88      print("Conexiune securizată")
89  else:
90      print("Conexiune nesecurizată")
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 16 ---
Conexiune securizată
```

FIG. 2.8 — https://

Verifica daca este conectiune securizata

```
95  print("\n--- Exercițiul 17 ---")
96  response_http = requests.get("http://cybercor.org", timeout=TIMEOUT)
97  for resp in response_http.history:
98      print(f"Redirecționare: {resp.status_code} -> {resp.url}")
99  print("URL-ul final după redirecționare:", response_http.url)
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 17 ---
Redirecționare: 301 -> http://cybercor.org/
URL-ul final după redirecționare: https://cybercor.org/
```

FIG. 2.9 — Istorie

Urmărirea și afișarea istoricului redirecționărilor HTTP

```
106  print("\n--- Exercițiul 18 ---")
107  response_head = requests.head(BASE_URL, timeout=TIMEOUT)
108  time.sleep(1)
109  response_get = requests.get(BASE_URL, timeout=TIMEOUT)
110
111  print("Lungime conținut HEAD:", len(response_head.content))
112  print("Lungime conținut GET:", len(response_get.content))
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 18 ---
Lungime conținut HEAD: 0
Lungime conținut GET: 1540
```

FIG. 2.10 — head & get

```
121  print("\n--- Exercițiul 19 ---")
122  if response.cookies:
123      for cookie in response.cookies:
124          print(f"Cookie nume: {cookie.name}, Secure: {cookie.secure}")
125  else:
126      print("Niciun cookie setat")
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 19 ---
Niciun cookie setat
```

FIG. 2.11 — cookie

Verifica daca sunt cookies

```python
print("\n--- Exercițiul 20 ---")
sesiune = requests.Session()
sesiune.headers.update({"User-Agent": "WebLab-<numele vostru>"})

response_sesiune = sesiune.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print("Rezultat de la ECHO_URL:\n", response_sesiune.text)
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 20 ---
Rezultat de la ECHO_URL:
 {
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, zstd",
    "Host": "httpbin.org",
    "User-Agent": "WebLab-<numele vostru>",
    "X-Amzn-Trace-Id": "Root=1-6ad63701-36282f111eb032e76468e848"
  }
}
```

FIG. 2.12 — Session

Utilizarea unui obiect `Session` din librăria `requests` pentru a seta un antet HTTP `User-Agent`

Capitolul III: 21-34

```python
def fetch(url, timeout=10):
    """Returnează obiectul răspuns pentru URL-ul dat."""
    return requests.get(url, timeout=timeout)

print("--- Exercițiile 21 & 23 ---")
print("Status Ex 21:", fetch(BASE_URL).status_code)
time.sleep(1)
print("Status Ex 23 (implicit):", fetch(BASE_URL).status_code)
time.sleep(1)
print("Status Ex 23 (timeout=3):", fetch(BASE_URL, timeout=3).status_code)
time.sleep(1)
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiile 21 & 23 ---
Status Ex 21: 200
Status Ex 23 (implicit): 200
Status Ex 23 (timeout=3): 200
```

FIG. 3.1 — Preluare

`fetch` pentru a efectua cereri HTTP GET cu parametri de timeout, testând și afișând codurile de stare ale răspunsurilor

```python
def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    response = fetch(url)
    return response.status_code

print("\n--- Exercițiul 22 ---")
for path in ["", "/robots.txt", "/sitemap.xml"]:
    print(f"Status pentru {path}:", get_status(BASE_URL + path))
    time.sleep(1)

print("\n--- Exercițiul 26 ---")
try:
    get_status(123)
except Exception as e:
    print("Eroare la apelarea cu 123:", type(e).__name__)
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiul 22 ---
Status pentru : 200
Status pentru /robots.txt: 200
Status pentru /sitemap.xml: 200

--- Exercițiul 26 ---
Eroare la apelarea cu 123: MissingSchema
```

FIG. 3.2 — Testare

Verifică statusul HTTP pentru mai multe rute web

```python
def get_title(html: str) -> str:
    """Extrage titlul dintr-un document HTML."""
    start = html.find("<title>")
    if start == -1:
        return "Fără titlu"
    start += len("<title>")
    end = html.find("</title>")
    return html[start:end].strip()

print("\n--- Exercițiile 24 & 25 ---")
html_test = fetch(BASE_URL).text
print("Titlu:", get_title(html_test))
help(get_title)  # Afișează docstring-ul din funcție
```

Problems Output Debug Console Terminal Ports

```
--- Exercițiile 24 & 25 ---
Titlu: CYBERCOR
Help on function get_title in module __main__:

get_title(html: str) -> str
    Extrage titlul dintr-un document HTML.
```

FIG. 3.3 — Extragere

extrage manual textul dintre etichetele `<title>` ale unui document HTML folosind metodele de căutare (`find()`) și decupare

```python
def page_exists(url: str) -> bool:
    """Verifică dacă o pagină poate fi descărcată fără erori."""
    try:
        response = requests.get(url, timeout=TIMEOUT)
        response.raise_for_status()
        return True
    except requests.RequestException:
        return False

print("\n--- Exercițiul 27 ---")
print("Există pagina validă?:", page_exists(BASE_URL))
time.sleep(1)
print("Există pagina invalidă?:", page_exists("https://this-domain-does-not-exist.invalid"))
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 27 ---
Există pagina validă?: True
Există pagina invalidă?: False
```

FIG. 3.4 — Verificare

verifică dacă o pagină web există și poate fi accesată fără erori, folosind metoda `raise_for_status()` alături de gestionarea excepțiilor (`try-except`)

```python
def check_paths(base: str, paths: list) -> dict:
    """Verifică o listă de căi și returnează un dicționar cu codurile de stare."""
    rezultate = {}
    for path in paths:
        rezultate[path] = get_status(base + path)
        time.sleep(1)
    return rezultate

print("\n--- Exercițiul 28 ---")
print("Căi verificate:", check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))
```

Problems Output Debug Console **Terminal** Ports

```
--- Exercițiul 28 ---
Căi verificate: {'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
```

FIG. 3.5 — Mapare

Parcurge printr-o listă de căi web asociate unui URL de bază

```python
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Afișează valoarea unui singur antet HTTP."""
    resp = fetch(url)
    return resp.headers.get(name, default)

print("\n--- Exercițiul 29 ---")
print("Server:", get_header(BASE_URL, name="Server"))
time.sleep(1)
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 29 ---
Server: cloudflare
```

FIG. 3.6 — Interogare

Extrage valoarea unui anumit antet HTTP

```python
def security_headers(url: str) -> dict:
    """Verifică prezența a 5 antete de securitate importante."""
    antete_cautate = [
        "Strict-Transport-Security", "Content-Security-Policy",
        "X-Frame-Options", "X-Content-Type-Options", "Referrer-Policy"
    ]
    resp = fetch(url)
    rezultate = {}
    for antet in antete_cautate:
        # Dacă răspunsul e True înseamnă că antetul există în headers
        rezultate[antet] = antet in resp.headers
    return rezultate

def score_headers(results: dict) -> str:
    """Calculează un scor de tip '3/5' pe baza antetelor găsite."""
    gasite = sum(results.values())
    total = len(results)
    return f"{gasite}/{total}"

print("\n--- Exercițiile 30 & 31 ---")
rezultat_securitate = security_headers(BASE_URL)
print("Analiză securitate:", rezultat_securitate)
print("Scor final:", score_headers(rezultat_securitate))
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiile 30 & 31 ---
Analiză securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options':
True, 'Referrer-Policy': True}
Scor final: 3/5
```

FIG. 3.7 — Evaluare

Verifică prezența a cinci antete HTTP de securitate esențiale pentru un URL

```python
def fetch_robots(base: str):
    """Descarcă și returnează textul din /robots.txt."""
    resp = requests.get(base + "/robots.txt", timeout=TIMEOUT)
    if resp.status_code == 200:
        return resp.text
    return None

def disallowed_paths(robots_text: str) -> list:
    """Extrage căile interzise din conținutul fișierului robots.txt."""
    if not robots_text:
        return []

    interzise = []
    for linie in robots_text.splitlines():
        if linie.startswith("Disallow:"):
            # Luăm textul de după "Disallow: "
            cale = linie.split(":", 1)[1].strip()
            interzise.append(cale)
    return interzise

print("\n--- Exercițiul 32 ---")
text_robots = fetch_robots(BASE_URL)
print("Căi blocate găsite:", disallowed_paths(text_robots))
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 32 ---
Căi blocate găsite: []
```

FIG. 3.8 — Parsare

descarcă conținutul unui fișier `robots.txt` de pe site și extrage o listă cu toate căile interzise

```python
def response_times(*urls):
    """Returnează un dicționar cu timpul de răspuns pentru mai multe URL-uri."""
    timpi = {}
    for url in urls:
        start = time.perf_counter()
        requests.get(url, timeout=TIMEOUT)
        timpi[url] = round(time.perf_counter() - start, 4)
        time.sleep(1)
    return timpi

print("\n--- Exercițiul 33 ---")
print("Timpi de răspuns:", response_times(BASE_URL, "https://cybercor.org/robots.txt"))
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 33 ---
Timpi de răspuns: {'https://cybercor.org': 0.1408, 'https://cybercor.org/robots.txt': 0.1389}
```

FIG. 3.9 — Cronometrare

Returneaza timpl de raspuns

```
182  def log(message, **details):
183      """Afișează un mesaj însoțit de detalii suplimentare."""
184      rezultat = message
185      for cheie, valoare in details.items():
186          rezultat += f" | {cheie}={valoare}"
187      print(rezultat)
188
189  print("\n--- Exercițiul 34 ---")
190  log("verificat", url=BASE_URL, status=200)
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 34 ---
verificat | url=https://cybercor.org | status=200
PS C:\py_proiecte>
```

FIG. 3.10 — Jurnalizare

O funcție care construiește și afișează un mesaj formatat

Capitolul V: 35-43

```
4   import time
5   import requests
6   import urllib.parse
7   import re
8   from html.parser import HTMLParser
9   import hashlib
10  import json
11  import socket
12  import ssl
13  from datetime import datetime
```

Fig. 4.1. — Biblioteci

```
26  print("--- Exercițiul 35 ---")
27  url_test = "https://cybercor.org/path?x=1#top"
28  parsed = urllib.parse.urlparse(url_test)
29  print(f"Scheme: {parsed.scheme}")
30  print(f"Netloc: {parsed.netloc}")
31  print(f"Path: {parsed.path}")
32  print(f"Query: {parsed.query}")
33  print(f"Fragment: {parsed.fragment}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 35 ---
Scheme: https
Netloc: cybercor.org
Path: /path
Query: x=1
Fragment: top
```

FIG. 4.2 — Descompunere

Descompune o adresa

```
38  print("\n--- Exercițiul 36 ---")
39  print("Absolut pentru /about:", urllib.parse.urljoin(BASE_URL, "/about"))
40  print("Absolut pentru contact.html:", urllib.parse.urljoin(BASE_URL, "contact.html"))
41  print("Absolut pentru ../index.html:", urllib.parse.urljoin(BASE_URL, "../index.html"))
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 36 ---
Absolut pentru /about: https://cybercor.org/about
Absolut pentru contact.html: https://cybercor.org/contact.html
Absolut pentru ../index.html: https://cybercor.org/index.html
```

FIG. 4.3 — Combinare

combină o adresă web de bază cu diverse căi relative

```
46  def extract_links(html_content: str) -> list:
47      """Extrage toate legăturile (href) dintr-un document HTML, fără duplicate."""
48      gasite = re.findall(r'href="([^"]+)"', html_content)
49      return list(set(gasite)) # transformarea în set elimină duplicatele
50
51  print("\n--- Exercițiul 37 ---")
52  toate_legaturile = extract_links(html)
53  print(f"S-au găsit {len(toate_legaturile)} legături unice. Primele 3: {toate_legaturile[:3]}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 37 ---
S-au găsit 1 legături unice. Primele 3: ['/assets/index-CErP0EKt.css']
```

FIG. 4.4 — Extragere

Expresii regulate pentru a găsi și extrage o listă cu toate legăturile

```python
def split_links(links: list, domain: str) -> tuple:
    """Împarte lista de legături în interne (același domeniu) și externe."""
    interne = []
    externe = []
    for link in links:
        # Formăm URL-ul complet mai întâi
        full_url = urllib.parse.urljoin("https://" + domain, link)
        # Extragem domeniul
        netloc = urllib.parse.urlparse(full_url).netloc

        if netloc == domain:
            interne.append(full_url)
        else:
            externe.append(full_url)

    return interne, externe

print("\n--- Exercițiul 38 ---")
linkuri_interne, linkuri_externe = split_links(toate_legaturile, "cybercor.org")
print(f"Legături interne: {len(linkuri_interne)}")
print(f"Legături externe: {len(linkuri_externe)}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 38 ---
Legături interne: 1
Legături externe: 0
```

FIG. 4.5 — Separare

Analizează o listă de adrese web și le împarte în două categorii

```python
print("\n--- Exercițiul 39 ---")
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # Dacă tag-ul este de tip imagine, îi extra
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)

test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
assert finder.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul de validare a parserului a trecut cu succes!")

finder_real = ImageFinder()
finder_real.feed(html)
print(f"Pe site-ul real au fost găsite {len(finder_real.images)} imagini.")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 39 ---
Testul de validare a parserului a trecut cu succes!
Pe site-ul real au fost găsite 0 imagini.
```

FIG. 4.6 — Extragere

definește un parser care analizează un document HTML pentru a identifica etichetele de tip imagine și a extrage sursele acestora (atributele `src`) într-o listă.

```python
def page_fingerprint(url: str) -> str:
    """Returnează hash-ul (amprenta digitală) a conținutului paginii."""
    resp = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(resp.content).hexdigest()

print("\n--- Exercițiul 40 ---")
hash1 = page_fingerprint(BASE_URL)
time.sleep(1)
hash2 = page_fingerprint(BASE_URL)
print(f"Sunt hash-urile identice? {hash1 == hash2}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 40 ---
Sunt hash-urile identice? True
```

FIG. 4.7 — Amprentare

Descarcă conținutul unei pagini web și calculează hash-ul SHA-256.

```python
print("\n--- Exercițiul 41 ---")
# Salvăm
with open("headers.json", "w", encoding="utf-8") as file:
    json.dump(dict(response.headers), file, indent=2)

# Încărcăm înapoi
with open("headers.json", "r", encoding="utf-8") as file:
    headers_incarcate = json.load(file)
    print(f"Antetul 'Content-Type' citit din JSON: {headers_incarcate.get('Content-Type')}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 41 ---
Antetul 'Content-Type' citit din JSON: text/html; charset=utf-8
```

FIG. 4.8 — Stocare

salvează anteturile unui răspuns HTTP într-un fișier format JSON, după care le citește înapoi pentru a extrage și afișa o valoare specifică

```python
def resolve(hostname: str) -> str:
    """Returnează adresa IP a unui hostname."""
    return socket.gethostbyname(hostname)

print("\n--- Exercițiul 42 ---")
print(f"Adresa IP pentru cybercor.org este: {resolve('cybercor.org')}")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 42 ---
Adresa IP pentru cybercor.org este: 172.67.211.96
```

FIG. 4.9 — IP

Converteste numele de domeniu in adresa IP

```python
def cert_days_left(hostname: str) -> int:
    """Se conectează la portul 443 și returnează zilele până la expirarea SSL."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            expire_date_str = cert["notAfter"]
            # Convertim data SSL în secunde
            expire_seconds = ssl.cert_time_to_seconds(expire_date_str)
            # Timpul curent în secunde
            current_seconds = time.time()
            zile = (expire_seconds - current_seconds) / (24 * 3600)
            return int(zile)

print("\n--- Exercițiul 43 ---")
print(f"Certificatul cybercor.org mai este valabil {cert_days_left('cybercor.org')} zile.")
```

```
Problems  Output  Debug Console  Terminal  Ports

--- Exercițiul 43 ---
Certificatul cybercor.org mai este valabil 44 zile.
```

FIG. 4.10 — Verificare

Codul se conectează la un server securizat pentru a obține detaliile certificatului SSL

Capitolul VI: 44–50

```python
import time
import json
import re

TIMEOUT = 10
# Exercițiul 47: Constante de modul
DEFAULT_HEADERS = {"User-Agent": "WebLab-Tîltu Daniel"}

def fetch(url, timeout=TIMEOUT):
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

def get_status(url):
    return fetch(url).status_code

def get_title(html):
    start = html.find("<title>")
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\py_proiecte\web_lab> python -u "c:\py_proiecte\web_lab\webtools.py"
Autotest: 200
```

FIG. 5.1 — web_tools

În această secțiune, am creat un modul personalizat denumit `webtools.py`, al cărui scop principal este de a grupa toate funcțiile de analiză web dezvoltate în exercițiile anterioare într-o singură bibliotecă reutilizabilă. Astfel, separăm logica de extragere a datelor de execuția principală a programului, respectând principiile de organizare modulară a codului.

```
PS C:\py_proiecte\web_lab> python main.py https://cybercor.org
Constanta importată (Ex 47): {'User-Agent': 'WebLab-Tîltu Daniel'}

Verificăm căile pentru a genera report.csv...
Fișierul report.csv a fost creat.

=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         104.21.91.71
Redirecționări:    301 -> http://cybercor.org/
Scor securitate:   3/5
Certificat:        44 de zile rămase
Legături:          1 interne, 0 externe
Căi interzise:     Niciuna
Salvat în report.json
```

FIG. 5.1 — Raportul

Fișierul `main.py` reprezintă punctul de intrare (entry point) al aplicației noastre. Scopul său este să folosească instrumentele pe care le-am definit în biblioteca `webtools.py` pentru a executa un flux de lucru complet, de la preluarea comenzii utilizatorului până la generarea rapoartelor finale. Acest fișier demonstrează cum un modul personalizat poate fi importat și utilizat într-un program principal.

CONCLUZII

În urma efectuării acestei lucrări de laborator, am asimilat conceptele fundamentale privind comunicarea web și organizarea profesională a codului în limbajul Python.

1. **Diferențierea și utilizarea modulelor:** Am exersat diverse stiluri de import, înțelegând distincția practică dintre modulele standard (ex: `urllib`) și bibliotecile externe (ex: `requests`).
2. **Extragerea datelor web:** Am învățat să manipulez cererile și răspunsurile HTTP, lucrând cu statusuri, antete de securitate, cookie-uri și parsare de text (HTML).
3. **Structurarea codului:** Am transformat scripturi secvențiale în funcții reutilizabile (utilizând docstring-uri, parametri impliciți și adnotări de tip).

4. **Dezvoltarea unui modul propriu:** Am integrat toate cunoștințele în dezvoltarea proiectului final, creând un modul personalizat (`webtools.py`). Acesta, apelat din scriptul principal (`main.py`), generează rapoarte complexe despre un site web și le salvează automat în formatele structurate `.csv` și `.json`.

BIBLIOGRAFIE

1. **Suportul de curs :** *Laborator Python: funcții, metode și importuri pe web*.
2. **Documentația oficială Python 3:** Standard Library (modulele `urllib`, `json`, `csv`, `socket`, `ssl`, `re`, `html.parser`, `argparse`), disponibilă online la: https://docs.python.org/3/library/
3. **Documentația oficială a bibliotecii Requests:** *Requests: HTTP for Humans*, disponibilă online la: https://requests.readthedocs.io/
4. **Mozilla Developer Network (MDN) Web Docs:** Pentru aprofundarea termenilor web (HTTP Status Codes, Headers, Request/Response body), disponibilă online la: https://developer.mozilla.org/